Agenda

1.	Einleitung	2
2.	Versionsverwaltung	18
3.	Betriebssysteme	49
a.	Dateisystem	70
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Wozu Versionsverwaltung?

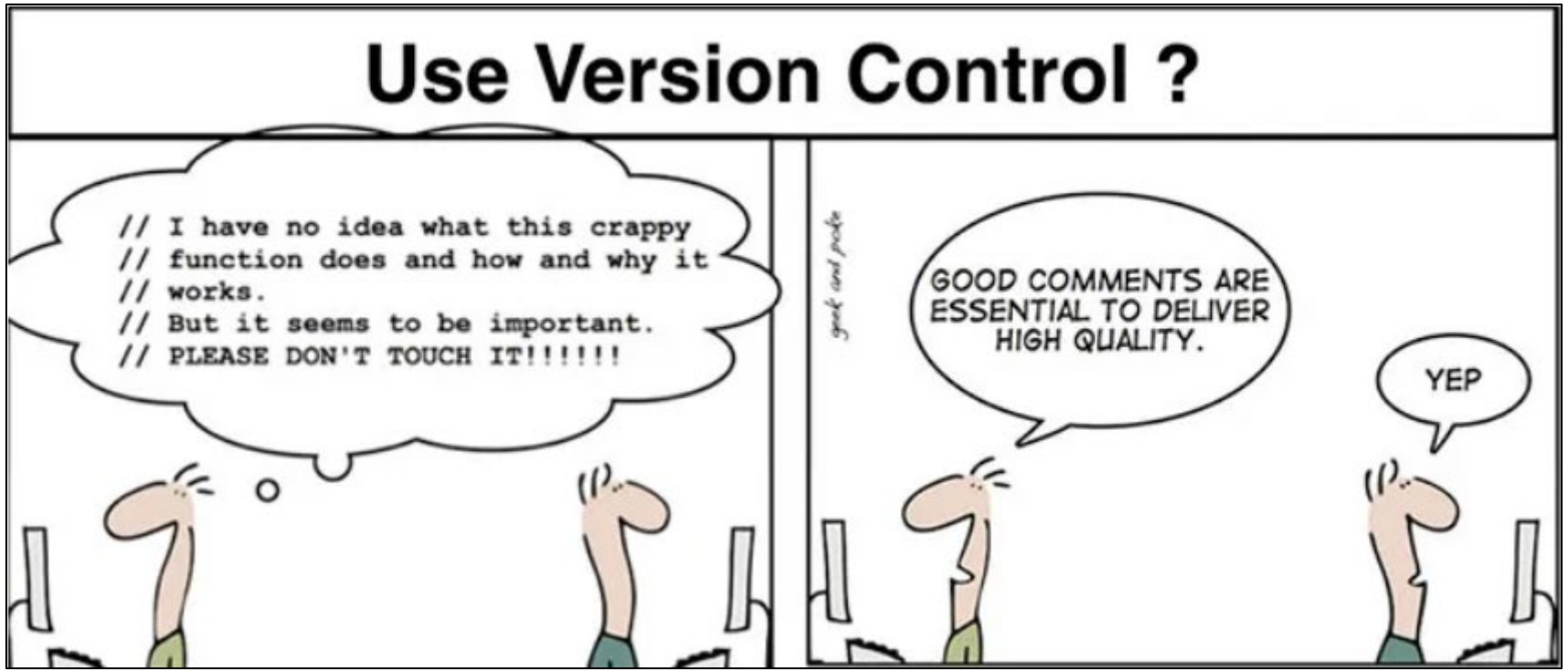
- Im Laufe eines Projektes entstehen verschiedene Artefakte, wie z.B. Source Code, Konfigurationsdateien, Dokumentation, …
- Eine Änderung funktioniert nicht … wie war nochmal der Stand gestern? Oder letzte Woche?
- Mehrere Personen arbeiten am gleichen Projekt … „Kannst Du mir mal Deine Version per Mail schicken“?
- „Meine Festplatte ist kaputt … hat noch jemand eine Kopie“?

Oft gesehen:

```
Main.java.3100125a
Main.java.010225a
Main.java.010225b
Main.java.bak
Main.java.orig
Main.java.works
…
```

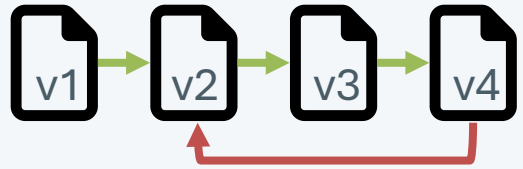
Geht das besser? Systematischer?

Qualität ist wichtig …

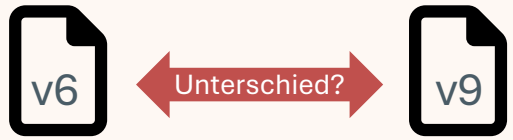


Ziele von Versionskontrolle

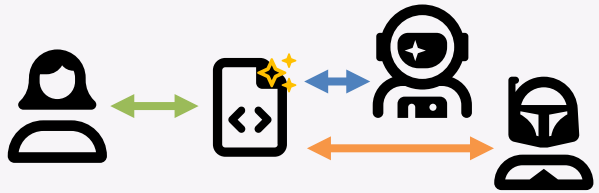
1. **Sicherung**
   Die Möglichkeit, ältere Versionen wiederherzustellen, um Fehler zu beheben oder unerwünschte Änderungen rückgängig zu machen.



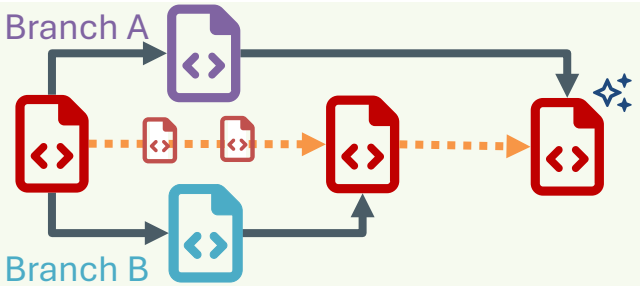
2. **Veränderungshistorie und Vergleich**
   Man kann Änderungen zwischen Versionen vergleichen, um Unterschiede zu erkennen.



3. **Kollaboration**
   Ermöglicht mehreren Entwicklern, gleichzeitig an einem Projekt zu arbeiten, ohne sich gegenseitig in die Quere zu kommen.

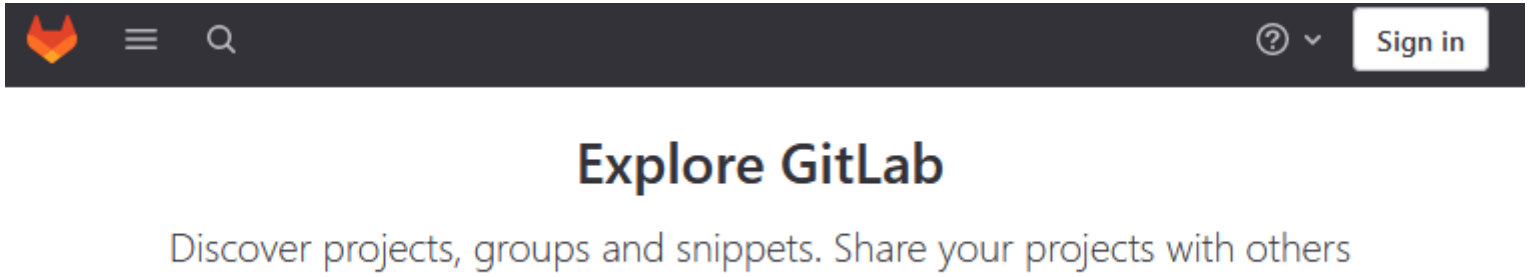


4. **Verzweigungen und Zusammenführung (Branching & Merging)**
   Arbeiten in verschiedenen Entwicklungszweigen, z. B. für neue Funktionen oder Bugfixes, ohne den Hauptcode zu beeinträchtigen.
   Späteres Zusammenführen der Änderungen ist möglich.



gitlab

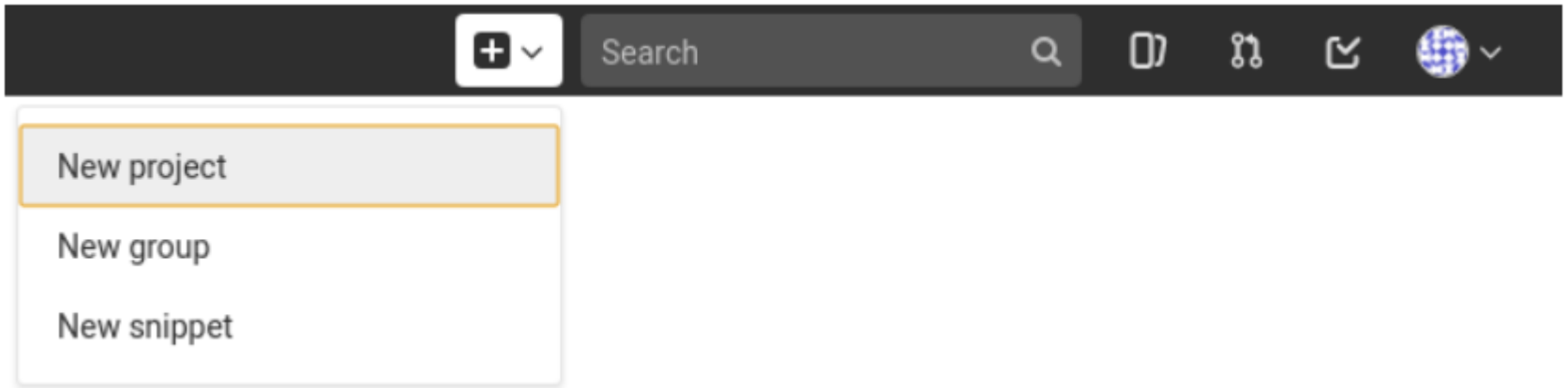
- Die THM (Fachbereich MNI) betreibt eine eigene Instanz von **gitlab**
- http://git.thm.de



- Login mit Ihrer THM-ID
- Generieren Sie einen SSH Key:
  ```
  ssh-keygen -t rsa -C "your.email@mni.thm.de" -b 4096
  ```
- Kopieren Sie den SSH Key in das Clipboard
- Klicken Sie auf git.thm.de auf Ihr Profil oben rechts, „Edit Profile“, „SSH Keys“, „Paste“
- Installieren Sie die „git bash“
- Mehr Infos: https://howto.git-pages.thm.de/

Anlegen eines Projektes

Nach dem Einloggen legen Sie ein neues Projekt an:



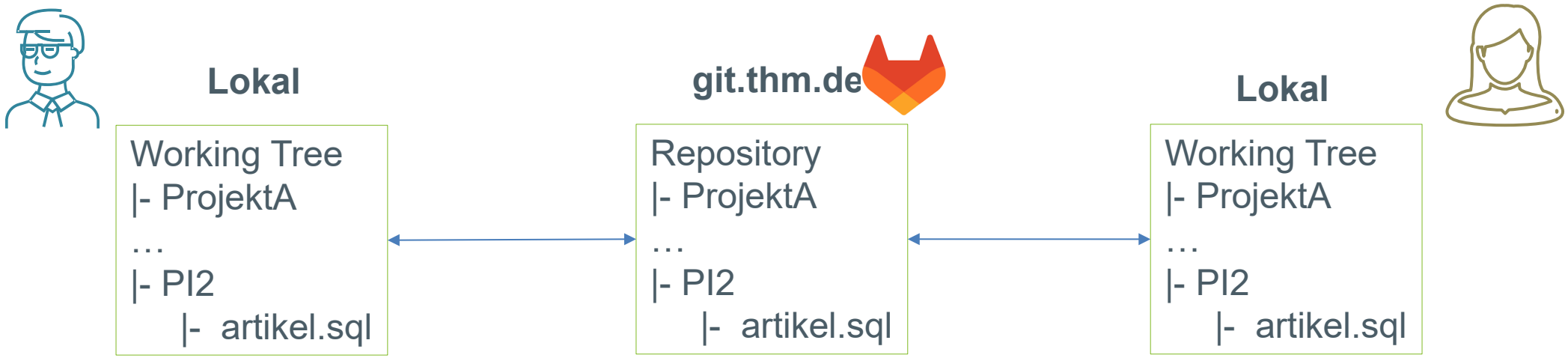
Projektnamen eintragen, „Private“ ankreuzen und „Create Project“.

Einmalig (nicht bei jedem neuen Projekt!) sollten Sie zwei Konfigurationen setzen (in der git bash):

```
git config --global user.name <name>
git config --global user.email <email>
```

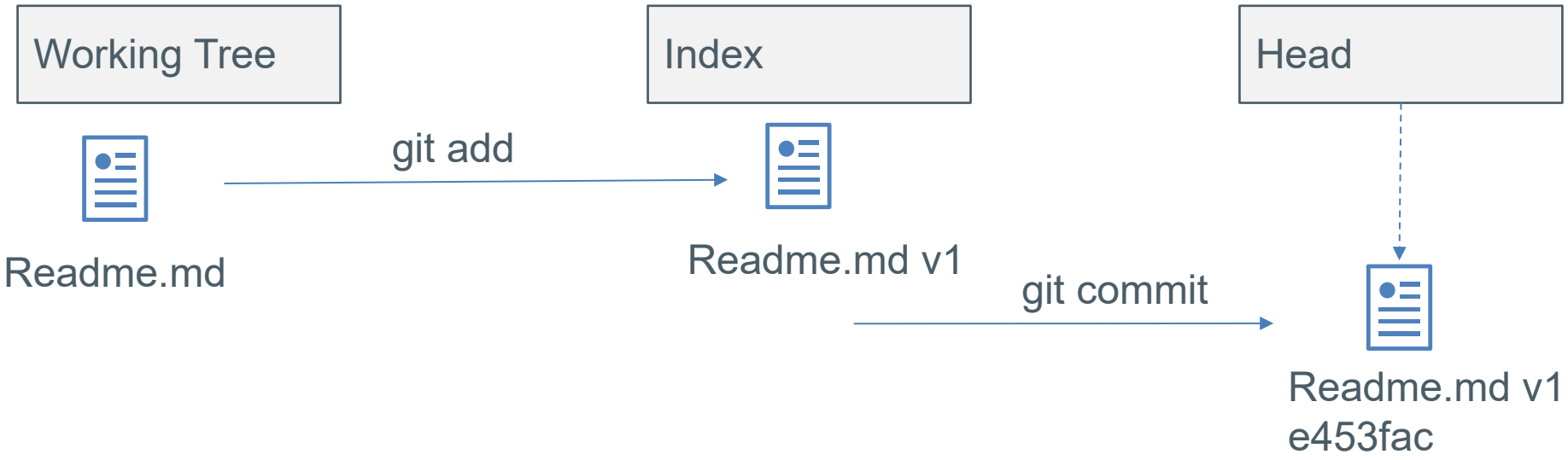
Clonen

- Aktuell existiert Ihr Projekt nur auf dem gitlab-Server.
- Klicken Sie im Projekt rechts auf „Code“ -> „Clone with SSH“, klicken Sie auf „Copy URL“
- Öffnen Sie „git bash“ und geben Sie ein:
  `git clone` gefolgt vom String, den Sie kopiert haben.
- Jetzt haben Sie eine Kopie des Projektes **lokal** auf Ihrem Rechner – in Ihrem **Working Tree**.



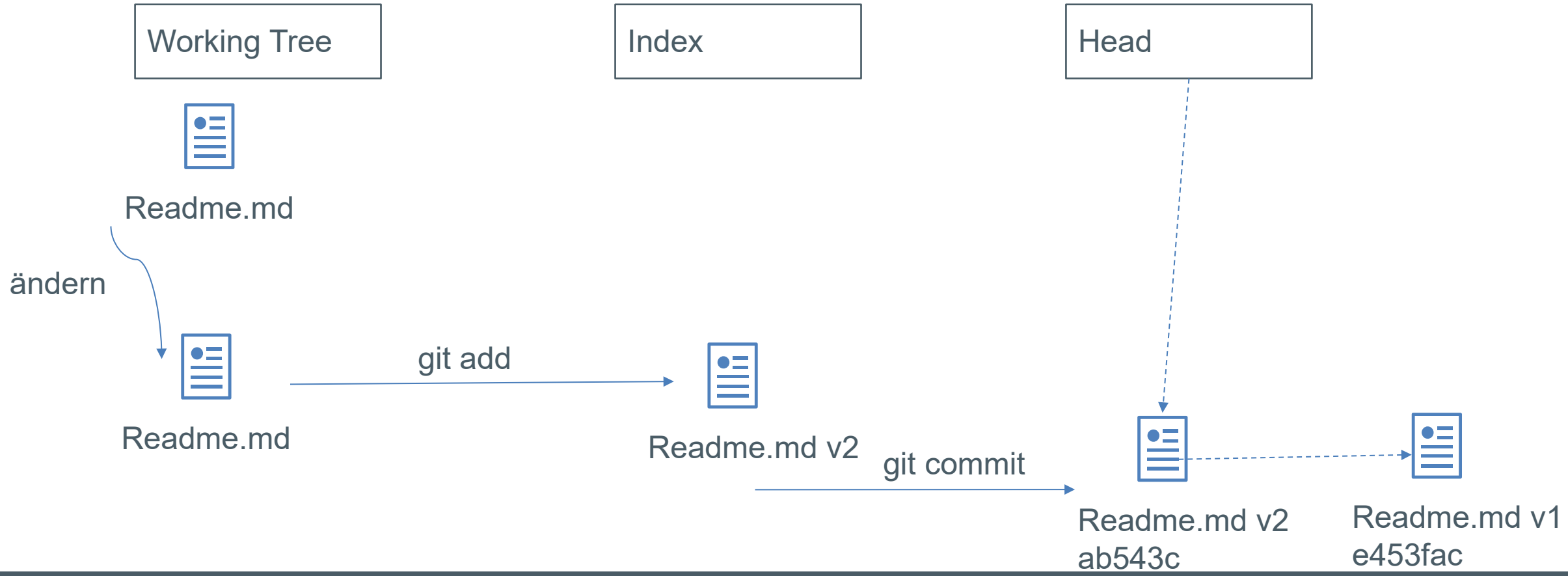
Working Tree, Index und Head (1)

In Wirklichkeit gibt es lokal drei Bereiche:



Working Tree, Index und Head (2)

In Wirklichkeit gibt es lokal drei Bereiche:



Demo

- Die langen Hexadezimalzahlen sind die **`COMMIT ID‘s.`**
- **`git status`** zeigt den Status der drei Bereiche an
- **`git log`** zeigt die Commits
- **`git diff`** zeigt Änderungen zwischen Versionen an
  Beispiel: `git diff HEAD^` zeigt die Änderung zur vorigen Version (`HEAD^=HEAD-1`)
- **`git checkout`** `setzt den HEAD auf einen bestimmten Commit.`
  Beispiele: `git checkout fa5cf3f82b7e37cbad04aefa692c0a875a5e0680`
  `git checkout master`

Rückgängigmachen von Änderungen

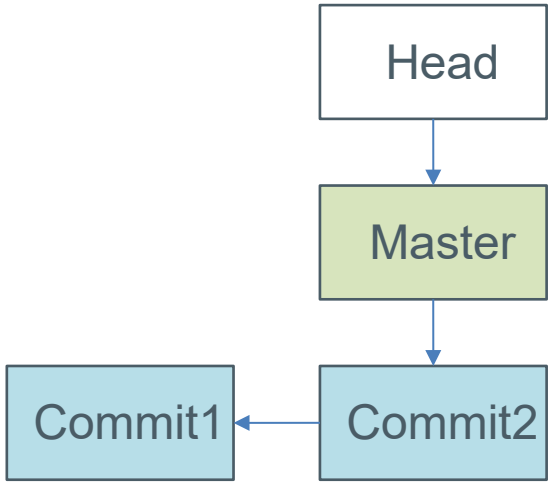
- Erste Methode:
  - **`git revert`** führt die gegenteiligen Änderungen eines Commits durch
    Beispiel:`git revert HEAD` erzeugt einen neuen Commit, der die letzten Änderungen rückgängig macht.

- Zweite Methode:
  - **`git reset`** setzt den HEAD auf einen vorigen Commit zurück

    `git restore` setzt dann das Working Directory auf dann den Stand, auf den HEAD zeigt, zurück.

Branching (1)

- Um an mehreren Versionen eines Projektes arbeiten zu können, nutzt man Branches („Zweige“)
- Der erste Branch ist immer der master Branch

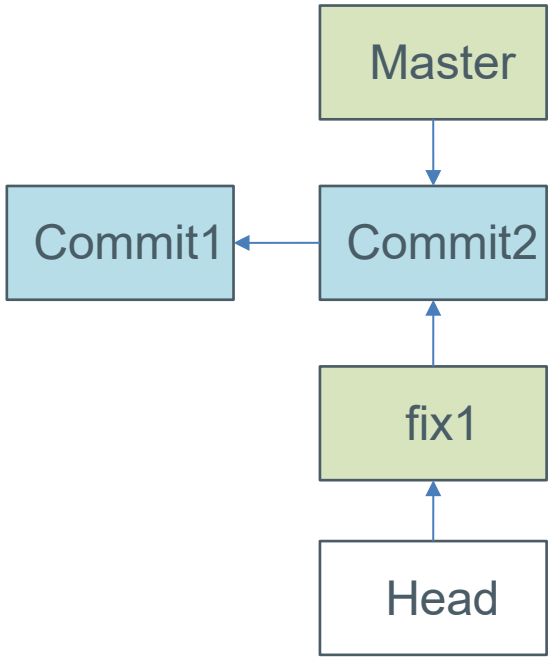


Branching (2)

- Um an mehreren Versionen eines Projektes arbeiten zu können, nutzt man Branches („Zweige“)
- Der erste Branch ist immer der master Branch

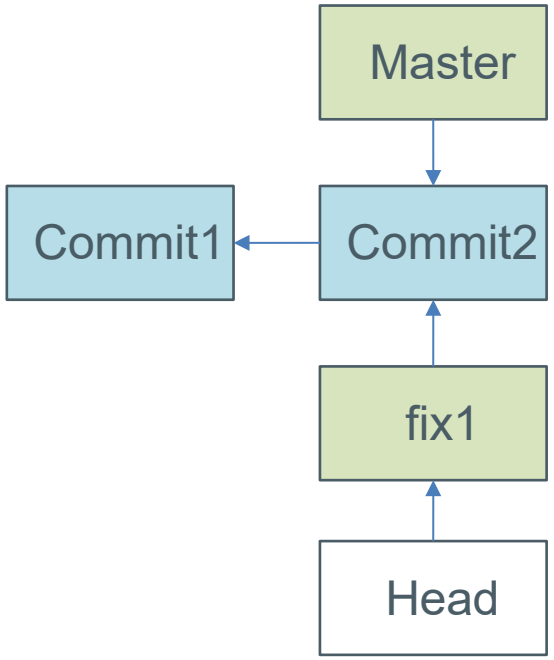
Mittels `git branch fix1` legt man einen neuen Branch fix1 an

Mittels `git checkout fix1` setzt man HEAD auf den neuen Branch



Branching (2)

- Um an mehreren Versionen eines Projektes arbeiten zu können, nutzt man Branches („Zweige“)
- Der erste Branch ist immer der master Branch



Mittels `git branch fix1` legt man einen neuen Branch fix1 an

Mittels `git checkout fix1` setzt man HEAD auf den neuen Branch

`git branch -a zeigt die Liste aller Branches`

Branching (3)

- Um an mehreren Versionen eines Projektes arbeiten zu können, nutzt man Branches („Zweige“)
- Der erste Branch ist immer der master Branch

- Nach einem weiteren Commit ändert sich der Branch fix1, aber nicht master:

